

Policy Beside the Data: An In-Database Control Plane for Agentic Workloads

Parag Paul
Agentic Memory Foundation
San Francisco, USA
parag@rigai.co

Abstract

AI agents invent SQL at runtime, chain tools through MCP, and often share a service account. Classical RLS answers *which rows*, not *which tool*, *in which session*, *after which prior acts*. Since March 2026, **Oracle Deep Data Security** in Oracle AI Database 26ai enforces identity-aware row/column grants at SQL rewrite time (Plane A). We argue the database is the source of truth, but two planes must stay distinct: Plane A (rows/columns) and Plane B (tools, session history, obligations). Oracle validates Plane A for agentic SQL; it does not cover tool orchestration, temporal quotas, soft guidance, or non-SQL MCP tools.

We survey Postgres/SQL Server RLS, Oracle VPD/RAS/Deep Data Security, Cedar, Dogwood, Rego, Zanzibar/OpenFGA, pgauthz, and MCP firewalls, and find **no open PostgreSQL extension** combining tool-native vocabulary, session-temporal constraints, soft guidance, and `CREATE EXTENSION` packaging. We present **pg.agent.policy** and **APL** for Plane B inside PostgreSQL. We disentangle per-row RLS cost (3–8× unindexed; ≈2% p95 indexed) from once-per-tool PDP cost (Python oracle 16–263 μs p50; reproducible PL/pgSQL via `bench.evaluate.pg.sh`; conservative envelope 0.8–1.5 ms, <0.2% of an 800 ms LLM loop). Prescription: index RLS or Oracle DATA GRANT for rows; evaluate APL for tools; never put tool policy on the hot row path.

Keywords

PostgreSQL, row-level security, agentic AI, authorization, policy languages, MCP

ACM Reference Format:

Parag Paul. 2026. Policy Beside the Data: An In-Database Control Plane for Agentic Workloads. In *Proceedings of Conference on Innovative Data Systems Research (CIDR '27)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIDR '27, Amsterdam, The Netherlands

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

PostgreSQL spent three decades optimizing for a human-shaped client. Agentic systems invert that contract: a prompt becomes schema discovery, multi-join SQL, CSV export, and a sibling agent’s tools—none reviewed by a developer. RAG already showed that relevance is not authorization; tool-using agents expand the surface further [5, 8].

Oracle’s March 2026 **Deep Data Security** in Oracle AI Database 26ai lands in the “database is the last referee” camp for SQL: `CREATE DATA GRANT` policies rewrite queries so agents cannot omit end-user predicates, with identity via `ORA_END_USER_CONTEXT` [21, 22]. That validates database-resident authorization for agentic workloads, but Oracle’s design is Plane A (row/column/cell isolation), not tool/session orchestration across MCP.

Two camps still argue past each other. One keeps policy in sidecars (OPA, Cedar) and MCP process filters, citing RLS planner surprises. The other notes that any check not in the server can be walked around. Both are half right. Per-row policy on unindexed columns *is* expensive. Agents *will* talk to the database. The mistake is treating “policy in the database” as one mechanism. We split planes:

- A — data isolation** Which rows exist? `GRANT` + RLS / VPD.
- B — agent/tool control** May this agent call this tool, given session history? `APL` / `pg.agent.policy.evaluate`
- C — model/content** Is the prompt/output allowed? External guardrails.

Plane B is what production MCP Postgres servers reinvent in Node (read-only transactions, row caps, DDL regexes, denial logs) [10, 12, 15]. Those are policies. They should be data next to RLS.

Contributions: (1) an updated survey including Oracle Deep Data Security (26ai); (2) APL and `pg.agent.policy` [6] as an open PostgreSQL artifact for Plane B; (3) latency and expected-loss analysis with Python-oracle and PL/pgSQL microbench scripts; (4) SQL examples and domain packs.

This is a systems/industry paper: an open PGXS extension and v0.1 measurements, not a claim that APL replaces Cedar’s SMT analyzer [16].

2 What agents need that RLS does not provide

Agents require tool identity (`execute_sql` vs. `export_csv`), argument constraints, session memory (budgets, approvals), soft guidance (obligations), graduated enforcement (`log_only`→`enforce`),

Table 1: Who covers agentic DB policy? (= yes, ~ = partial.)

System	In-DB	Tools	Temporal	Guidance
PG / SQL Server RLS				
Oracle VPD / RAS				
Oracle Deep Data Security (26ai)				
Cedar				
Dogwood				
OPA/Rego	sidecar	~	~	~
Zanzibar / OpenFGA	store	~		
pgauthz				
MCP regex firewalls	process			~
pg_agent_policy				

and audit that names a human *and* an agent [4]. RLS cannot express “this MCP tool may not run DDL” without putting that predicate on the *row* path—the design error the performance camp rightly fears.

3 Survey: is there an equivalent?

Short answer: Oracle covers Plane A for agentic SQL on 26ai; no open Postgres extension covers Plane B.

Plane A (in-DB rows/columns). PostgreSQL RLS [26]; Oracle VPD/RAS [23]; **Oracle Deep Data Security (26ai)**: **CREATE DATA GRANT**, **ORA_END_USER_CONTEXT**, engine rewrite for identity-aware agent workloads [20–22]; SQL Server RLS [19]; IBM LBAC. Deep Data Security modernizes VPD for agents: end users, data roles, declarative row/column grants, workload-specific rules, controlled privilege elevation. Select AI + MCP case studies filter SQL by authenticated end-user identity. None speak tools, session temporal quotas, or soft guidance.

Policy-as-code sidecars. OPA/Rego [13]; Cedar [16]; Dogwood (Cedar + temporal, agent sequences) [9]; Sentinel (graded modes) [1]; Oso Polar (can emit SQL fragments) [24]. Dogwood is the closest *linguistic* relative; it is not **CREATE EXTENSION**.

Relationship engines. Zanzibar/SpiceDB/OpenFGA [7, 14, 25] excel at “user is viewer of document” and often *use* Postgres as a store while remaining a second operational plane.

Postgres cousins. pgauthz + optional pg_cel (in-PG ABAC) [2, 17]; pg_command_fw (DDL GUC firewall) [27]. Closest priors; neither is an agent-native language with guidance, temporal quotas, and packs.

MCP servers. After **COMMIT**; **DROP** bypasses in the deprecated reference Postgres MCP, serious servers independently reimplemented Plane B in-process. That duplication is the industrial smell.

4 Layered guardrails

Listing 1: Plane A: indexed RLS (row path).

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_iso ON orders
  USING (tenant_id = current_setting('app.tenant_id', true))
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true));
CREATE INDEX ON orders (tenant_id);
```

Table 2: APL matcher latency (in-process oracle, not SPI).

Policies	p50 (μs)	p95 (μs)	p99 (μs)
3	16.0	59.6	136.7
25	43.4	120.8	187.8
50	83.3	246.9	565.6
200	263.4	472.0	796.3

Listing 2: Plane B: once-per-tool APL (not a row qual).

```
CREATE EXTENSION pg_agent_policy;
SELECT pg_agent_policy.set_setting('enforcement_mode', 'log_only');
SELECT pg_agent_policy.upsert_policy('block_ddl', $apl$
forbid
  principal agent "langgraph:analytics"
  action tool "execute_sql"
  when { context.statement_type in ["DROP","TRUNCATE","ALTER","CREATE"] }
  reason "No_DDL"
$apl$);
SELECT pg_agent_policy.evaluate(
  'agent','langgraph:analytics','tool','execute_sql',
  'table','public.orders',
  '{"statement_type":"DROP","acting_for":"user:42","tenant_id":"acme"}'::
    jsonb,
  'thread-abc');
```

Invariant: do not implement Plane B as **USING** clauses. Fail-closed PEP sentinels: missing **acting_for/tenant_id**→**"unset"**; missing **approved**→**"false"**. If **evaluate()** is skipped: read-only role, **default_transaction_read_only**, no **BYPASSRLS**.

APL v0.1 is dollar-quoted because extensions cannot extend **gram.y**. Effects **permit/forbid/guide**; deny-overrides; modes **log_only/guide/enforce**. Domain packs (analytics, support, fintech, healthcare, devops, multi-agent) load via SQL after a universal baseline.

5 Slowdown objection

5.1 Two costs

RLS injects security quals into the plan [11]. Unindexed policy columns yield 3–8× pgbench regressions [18]; wrapping **auth.uid()** as (**SELECT auth.uid()**) is mandatory. With an index, a 1M-row study reports ≈2% p95 versus an equivalent manual **WHERE** [28]. That literature describes *row* policy.

evaluate() is specified to run *once per tool*. Putting it in a per-row SQL function would be operator error.

5.2 Experiment A: matcher microbench (Python oracle)

A Python oracle of v0.1 matching (globs, **eq/in**, temporal counts, deny-overrides) on the baseline pack padded with non-matching policies, 8000 iterations (CPython 3.9):

We quote a conservative PostgreSQL envelope for v0.1 on the *same connection*: 0.8 ms **evaluate**+**log**, 1.5 ms with extra audit. **experiments/bench_evaluate_pg.sh** reproduces PL/pgSQL wall time (results in **evaluate_pg_microbench.json**). Roadmap pgrx/Cedar targets microsecond-class eval [16].

5.3 Experiment B: versus the agent loop

An 800 ms model+tool step dwarfs PDP cost: 0.8 ms is 0.10% of the loop; a cross-AZ OPA hop (~9 ms) is 1.1%. User-perceived latency is the model.

5.4 Experiment C: expected loss vs. latency tax

Sensitivity model (**not** actuarial): 10k tool calls/day, 4 attempted-incident opportunities/year, \$250k fully loaded if a bypass is realized. Bypass probabilities are ordinal (app filters > regex MCP > sidecar > evaluate+RLS residual superuser misconfig). Compute tax of extra milliseconds at \$0.12/CPU-hour is noise (<\$2/yr). Expected loss spans \$80k (app-only at 8% P(bypass)) to \$100 (evaluate+RLS at 0.01%). Even if app-only P(bypass) is 1%, \$10k/yr still dominates. The industrial claim is the *inequality*, not a specific ROI.

5.5 When in-DB policy does hurt

Unindexed RLS; many OR-combined permissive policies; non-LEAKPROOF functions blocking pushdown; calling `evaluate()` per row; synchronous HTTP from triggers. `pg_agent_policy`'s API is a tool-level function, not a row trigger.

6 Value that is not latency

Co-location yields one trust boundary, one tenant GUC for RLS and APL, session events without Redis, SQL-queryable `decision_log`, pack portability, and gateway independence. The database is the source of truth because the rows, tenants, and side effects already live there—not because PL/pgSQL beats Cedar on a microbenchmark.

7 Threats to validity

Python oracle $\neq$ PL/pgSQL wall time (use `bench_evaluate_pg.sh`; envelope until CI installcheck). Oracle Deep Data Security enforces Plane A at rewrite time without a PEP call; `pg_agent_policy` v0.1 requires `evaluate()`—`ProcessUtility_hook` is roadmap. P(bypass) is assumed ordinal sensitivity. v0.1 lacks Dogwood's **formerly/since** and Cedar's SMT. Superusers bypass RLS. Managed allowlists (Neon, RDS) block arbitrary extensions; Oracle DDS is 26ai-only [3].

8 Conclusions

Agents make the database a tool. Oracle Deep Data Security confirms identity-aware authorization must live in the database for agentic SQL (Plane A). Index RLS or Oracle DATA GRANT for rows. Evaluate APL once per tool for Plane B (tools, temporal, guidance, non-SQL MCP). Do not accept “the database is too slow for policy” without asking *which plane*. Artifact: <https://github.com/rahiakil/pg-policy>.

Acknowledgments

Working paper of the Agentic Memory Foundation. `pg_agent_policy` is open source under the PostgreSQL License.

References

- [1] [n.d.]. HashiCorp Sentinel. <https://developer.hashicorp.com/sentinel>.
- [2] [n.d.]. `pg_cel`. <https://github.com/thomasdarimont/pgauthz/tree/main/extensions/pg-cel>.
- [3] [n.d.]. PostgreSQL Extension Network. <https://pgxn.org/>.
- [4] 2024. Regulation (EU) 2024/1689 of the European Parliament and of the Council (AI Act).
- [5] 2026. Securing the Agent: Vendor-Neutral, Multitenant Enterprise Retrieval and Tool Use. In *ACM*. DOI 10.1145/3786335.3813145.
- [6] Agentic Memory Foundation and Parag Paul. 2026. `pg_policy`: Agentic policy language for PostgreSQL. <https://github.com/rahiakil/pg-policy>.
- [7] AuthZed. [n.d.]. SpiceDB. <https://github.com/authzed/spicedb>.
- [8] AWS. 2026. Building agentic AI applications with a modern data mesh strategy on AWS. <https://aws.amazon.com/blogs/machine-learning/building-agentic-ai-applications-with-a-modern-data-mesh-strategy-on-aws/>.
- [9] AWS Open Source. 2026. Introducing Dogwood: runtime verification for AI agents. <https://aws.amazon.com/blogs/opensource/introducing-dogwood-runtime-verification-for-ai-agents/>.
- [10] Azzaraell. [n.d.]. `pgguard-mcp`. <https://github.com/Azzaraell/pgguard-mcp>.
- [11] Bytebase. [n.d.]. Postgres Row-Level Security (RLS) Limitations and Alternatives. <https://www.bytebase.com/blog/postgres-row-level-security-limitations-and-alternatives/>.
- [12] Samuel Cabral. [n.d.]. `safe-postgres-mcp`. <https://github.com/samuel-cabral/safe-postgres-mcp>.
- [13] CNCF. [n.d.]. Open Policy Agent. <https://www.openpolicyagent.org/>.
- [14] CNCF. [n.d.]. OpenFGA. <https://openfga.dev/>.
- [15] Crystal DBA. [n.d.]. `postgres-mcp`. <https://github.com/crystaldba/postgres-mcp>.
- [16] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo Heule, Kesha Hietala, John Koenig, K. Rustan M. Leino, Jacob R. Lorch, Bryan McMillan, Oded Padon, et al. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024).
- [17] Thomas Darimont. [n.d.]. `pgauthz`. <https://github.com/thomasdarimont/pgauthz>.
- [18] Jake. 2026. Supabase Row Level Security Policy Performance Overhead `pgbench`. <https://jakeinsight.com/tech/2026-05-09-supabase-row-level-security-policy-performance-over/>.
- [19] Microsoft. [n.d.]. Row-Level Security. <https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security>.
- [20] Oracle. 2026. Create Data Grants. <https://docs.oracle.com/en/database/oracle/oracle-database/26/ddscg/create-data-grants.html>.
- [21] Oracle. 2026. Introducing Oracle Deep Data Security: Identity-Aware Data Access Control for Agentic AI in Oracle AI Database 26ai. <https://blogs.oracle.com/database/introducing-oracle-deep-data-security-identity-aware-data-access-control-for-agentic-ai-in-oracle-ai-database-26ai>.
- [22] Oracle. 2026. Oracle Deep Data Security for Identity Aware Authorization in Autonomous AI Database. <https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsb/deep-data-security.html>.
- [23] Oracle. 2026. Using Oracle Virtual Private Database to Control Data Access. <https://docs.oracle.com/en/database/oracle/oracle-database/26/dbseg/using-oracle-vpd-to-control-data-access.html>.
- [24] Oso. [n.d.]. The Polar Language. <https://www.osohq.com/docs/oss/learn/polar-foundations.html>.
- [25] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dean, Jacob Gregorio, Andrew Gupta, Yuri Hsieh, Sebastian Kanthak, Eric Pivonka, et al. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *USENIX ATC*.
- [26] PostgreSQL Global Development Group. [n.d.]. Row Security Policies. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>.
- [27] rustwizard. [n.d.]. `pg_command.fw`. <https://github.com/rustwizard/pg-command.fw>.
- [28] Ashwin Sridhar. 2026. Does Postgres RLS actually ruin performance? Let's look at the data. https://dev.to/ashwin_sridhar_koto7/does-postgres-rls-actually-ruin-performance-lets-look-at-the-data-24jf.

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.